

INTERVIEW ASSIGNMENT: PROMPT-ONLY CONSTRAINT-BASED LEVEL GRAMMAR (TOP-DOWN)

GOAL

Design a prompt-driven system that generates a valid top-down level layout satisfying strict semantic and spatial constraints. All generation must be done via prompts. Output must be deterministic, structured, and validated within a strict runtime limit.

WHAT YOU NEED TO BUILD

A small web demo that: 1. Uses an LLM to generate a top-down level layout from a prompt 2. Outputs strict JSON only 3. Renders the layout using cubes only (three.js or equivalent) 4. Validates that all semantic constraints are satisfied

SCOPE LIMITATIONS

- Top-down view only
- Cubes only
- Grid-based layout
- No character controller required
- No textures, lighting, or advanced visuals
- No procedural generation logic replacing the model

LAYOUT TYPE

A dungeon-style layout based on a layout grammar with rooms and corridors that have gameplay meaning.

REQUIRED ELEMENTS (EXACT)

- 1 Spawn room
- 1 Exit room
- 1 Key room
- 1 Locked door
- 1 Optional Treasure room
- 2–4 Corridor segments
- Total rooms: 4–6

HARD SEMANTIC CONSTRAINTS

1. Key room must be reachable before the locked door
2. Locked door must block the shortest path to the exit
3. Treasure room must be optional and not on the critical path
4. All rooms must be connected
5. At most one optional loop

GEOMETRY CONSTRAINTS

- Grid size: 10×10 to 25×25
- Integer grid coordinates only
- Rectangular rooms
- Orthogonal corridors
- No overlapping rooms

DETERMINISM & CONTROL

- seed (integer)
- difficulty (1–5)
- density (1–5)

Same inputs must produce functionally identical layouts.

PROMPT RUNTIME CONSTRAINT

- Maximum total prompt execution time: 10 seconds
- Includes generation, self-check, and repair
- Must fail gracefully if constraints are not met within time

VISUALIZATION REQUIREMENTS

- Top-down camera
- Rooms rendered as cube groups
- Corridors rendered as cube paths
- Distinct colors for room types, doors, and corridors

VALIDATION REQUIREMENTS

- Build a graph from the output
- Verify all semantic constraints
- Detect violations and trigger repair or regeneration
- Validation may not redesign the layout

PROMPTING REQUIREMENTS

- Prompt must enforce planning before generation
- Strict schema enforcement
- Explicit self-check for every constraint
- Minimal repair on failure

ALLOWED

- three.js or equivalent
- JSON parsing
- Graph validation
- Simple UI for generation parameters

DISALLOWED

- Procedural layout generation replacing the model
- Manual editing of output
- Hard-coded valid layouts

DELIVERABLES

1. Web demo rendering the generated layout
2. Prompt pack (system, user, repair)
3. Short written explanation of approach

TIME EXPECTATION

3–5 hours

OUTPUT FORMAT (JSON ONLY)

```
{ "metadata": { "seed": 42, "difficulty": 3, "density": 2, "gridSize": [20, 20] },
  "rooms": [ { "id": "spawn", "type": "spawn", "pos": [2, 2], "size": [4, 4] } ],
  "corridors": [ { "from": "spawn", "to": "key", "path": [[6,4],[7,4],[8,4]] } ],
  "doors": [ { "id": "door_1", "type": "locked", "blocks": ["spawn", "exit"] } ],
  "selfCheck": { "keyBeforeDoor": true, "doorBlocksCriticalPath": true,
  "treasureOptional": true, "connected": true, "loops": 1, "notes": [] } }
```